



Escola de Engenharia
Programa de Pós-Graduação em Engenharia Elétrica

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

LISTA 6

Belo Horizonte
28 de junho de 2026

SUMÁRIO

1	PARTE 1	3
1.1	Exercício 1	3
1.2	Exercício 2	3
1.3	Exercício 3	5
1.4	Exercício 4	5
1.5	Exercício 5	5
1.6	Exercício 6	6
1.7	Exercício 7	8
1.8	Exercício 8	9
1.9	Exercício 9	10
2	PARTE 2	14
2.1	Exercício 1	14
2.2	Exercício 2	15
2.3	Exercício 3	16
ANEXO A	CÓDIGO EXERCÍCIO 1.6	18
ANEXO B	CÓDIGO EXERCÍCIO 1.8	21
ANEXO C	CÓDIGO EXERCÍCIO 2.1	25
ANEXO D	CÓDIGO EXERCÍCIO 2.2	29
ANEXO E	CÓDIGO EXERCÍCIO 2.3	33

1 PARTE 1

1.1 Exercício 1

Considere uma lei de conservação genérica dada por

$$\frac{\partial u}{\partial t} + \frac{\partial f}{\partial x} = 0$$

Aproximamos u e f por u_h e f_h , o que resulta em um Resíduo R dado por

$$R = \frac{\partial u_h}{\partial t} + \frac{\partial f_h}{\partial x}$$

Usando a técnica dos resíduos ponderados, temos que em cada elemento k

$$\int_{D^k} R \cdot v \, dx = 0$$

Fazendo v como a função de Lagrange e substituindo a expressão de R , temos

$$\int_{D^k} \left(\frac{\partial u_h}{\partial t} + \frac{\partial f_h}{\partial x} \right) L_j \, dx = 0$$

$$\int_{D^k} \frac{\partial u_h}{\partial t} L_j \, dx + \int_{D^k} \frac{\partial f_h}{\partial x} L_j \, dx = 0$$

Aplica integral por partes na segunda parcela da soma

$$\int_{D^k} \frac{\partial u_h}{\partial t} L_j \, dx = \int_{D^k} \frac{dL_j}{dx} f \, dx - [L_j f]_{x_L}^{x_R} = 0$$

Obtemos a forma fraca do esquema. Aplicando integração por partes uma segunda vez, obtemos a forma forte:

$$\int_{D^k} \frac{\partial u_h}{\partial t} L_j \, dx = [L_j f]_{x_L}^{x_R} - \int_{D^k} \frac{df}{dx} L_j \, dx - [L_j f]_{x_L}^{x_R} = 0$$

$$\int_{D^k} \frac{\partial u_h}{\partial t} L_j \, dx + \int_{D^k} \frac{\partial f}{\partial x} L_j + [L_j (f^* - f)]_{x_L}^{x_R} = 0$$

1.2 Exercício 2

Dentro de um elemento D^k , aproximamos $u(x, t)$ por uma interpolação por polinômios de Lagrange (nodal)

$$u(x, t) = u_h(x, t) = \sum_{i=1}^{N_p} u_i(t) L_i(x)$$

Logo,

$$\frac{\partial u_h}{\partial t} = \sum_{i=1}^{N_p} \frac{du_i}{dt} L_i$$

Substitui isso na forma fraca

$$\int_{D^k} \sum_{i=1}^{N_p} \frac{du_i}{dt} L_i L_j dx = \int_{D^k} \frac{dL_j}{dx} f dx - [L_j f]_{x_L}^{x_R} = 0$$

$$\sum_{i=1}^{N_p} \frac{du_i}{dt} \int_{D^k} L_i L_j dx = \int_{D^k} \frac{dL_j}{dx} f dx - [L_j f]_{x_L}^{x_R} = 0$$

Definindo

$$M_{ij} = \int_{D^k} L_i L_j dx$$

Temos

$$M \frac{du}{dt} = \dots$$

Agora analisamos a segunda integral da forma fraca. Substituindo $f = au$, temos

$$\int_{D^k} \frac{dL_j}{dx} au dx$$

Substituindo u_h ,

$$\int_{D^k} \frac{dL_j}{dx} a \sum_{i=1}^{N_p} u_i(t) L_i(x) dx$$

$$a \sum_{i=1}^{N_p} u_i(t) \int_{D^k} \frac{dL_j}{dx} L_i(x) dx$$

Definindo

$$S_{ij} = \int_{D^k} \frac{dL_j}{dx} L_i(x) dx$$

Temos

$$M \frac{d\mathbf{u}}{dt} = aS\mathbf{u} - \mathbf{F}$$

Usar a representação nodal é mais conveniente do ponto de vista de código uma vez que armazenamos os pontos (nós) em que a função u_h é avaliada.

1.3 Exercício 3

Quando $\alpha = 1$ (fluxo central), a segunda parcela se anula e temos apenas a média aritmética das bordas de ambos elementos vizinhos.

Quando $\alpha = 0$ (fluxo upwind), o fluxo leva em consideração a direção de propagação por meio dos termos $\hat{n}^+$ e $\hat{n}^-$.

1.4 Exercício 4

Os pontos GLL são escolhidos ao invés dos pontos equidistantes pois ele apresentam melhor desempenho no método numérico. Além disso, a matriz M se torna aproximadamente diagonal ao escolher os pontos GLL, o que melhora o desempenho computacional.

Os pontos GLL são tabelados, tornando fácil seu uso.

1.5 Exercício 5

Uma vez obtido a forma semi-discreta com o DGTD:

$$\frac{d\mathbf{u}}{dt} = \mathbf{R}(\mathbf{u})$$

usamos o método de Runge-Kutta explícito de quarta ordem de baixo armazenamento (Low-Storage Runge-Kutta – LSRK4). Diferentemente do RK4 clássico, esse algoritmo necessita armazenar apenas duas variáveis de trabalho, reduzindo significativamente o consumo de memória.

Definindo $\mathbf{r}$ como o vetor residual e Δt como o passo de tempo, o algoritmo é dado por

$$\mathbf{r}^{(i)} = a_i \mathbf{r}^{(i-1)} + \Delta t \mathbf{R}(\mathbf{u}^{(i-1)}), \quad (1.1)$$

$$\mathbf{u}^{(i)} = \mathbf{u}^{(i-1)} + b_i \mathbf{r}^{(i)}, \quad i = 1, \dots, 5, \quad (1.2)$$

onde $\mathbf{u}^{(0)} = \mathbf{u}^n$, $\mathbf{u}^{(5)} = \mathbf{u}^{n+1}$ e $\mathbf{r}^{(0)} = \mathbf{0}$.

E os coeficientes a e b são dados por

$$a = \left[0, -\frac{567301805773}{1357537059087}, -\frac{2404267990393}{2016746695238}, -\frac{3550918686646}{2091501179385}, -\frac{1275806237668}{842570457699} \right], \quad (1.3)$$

$$b = \left[\frac{1432997174477}{9575080441755}, \frac{5161836677717}{13612068292357}, \frac{1720146321549}{2090206949498}, \frac{3134564353537}{4481467310338}, \frac{2277821191437}{14882151754819} \right]. \quad (1.4)$$

O algoritmo completo pode ser descrito pelo seguinte pseudocódigo:

```
r = 0
u = u^n
```

```

for i = 1, ..., 5
    r = a(i)*r + dt*R(u)
    u = u + b(i)*r
end

u^(n+1) = u

```

Esse esquema preserva a precisão de quarta ordem no tempo, apresentando custo computacional semelhante ao RK4 clássico, porém com menor demanda de memória. Essa característica o torna particularmente adequado para métodos de alta ordem, como o DGTD, nos quais o número de graus de liberdade é elevado.

1.6 Exercício 6

Todo o código usado nesse exercício encontra-se no Anexo A.

Usando o método do DGTD para a equação da advecção 1D e os seguintes parâmetros de discretização:

- $K = 64$
- $N = 4$
- $\Delta t = 1.25 \cdot 10^{-4} \text{ s}$ (CFL = 0.2)
- $\Delta x = 1.5 \cdot 10^{-2} \text{ m}$
- $T = 1 \text{ s} \implies N_t = 8000$

podemos simular um período completo de uma advecção 1D com velocidade $a = 1$. O resultado obtido encontra-se na Figura 1. Vemos que o sinal se propaga para a direita e retorna para a posição inicial após um período completo, com a condição de contorno periódica.

Monitorando a posição valor máximo do ponto ao longo do tempo, podemos traçar sua posição ao longo do tempo e assim calcular a velocidade de propagação numérica para os fluxos central e upwind exibido na Figura 2.

Em ambos casos, a inclinação da reta na Figura 2 nos retorna a velocidade de propagação numérica e, a partir de uma regressão linear simples, obtemos $a_{NUM} = 1.009 \text{ m/s}$, com um erro de apenas 0.9 % em relação ao valor analítico, sendo esse um indicador do funcionamento da implementação.

Uma outra maneira de testar a implementação é comparar com a solução analítica, dada por

$$u_{exact}(x, t) = u_0(x - at) = \sin(2\pi(x - t))$$

através da norma- L^2 e analisar a convergência conforme aumentamos o número de elementos K no domínio. O resultado da convergência está exibido na Figura 3.

Figura 1 – Propagação da advecção 1D ao longo do tempo.

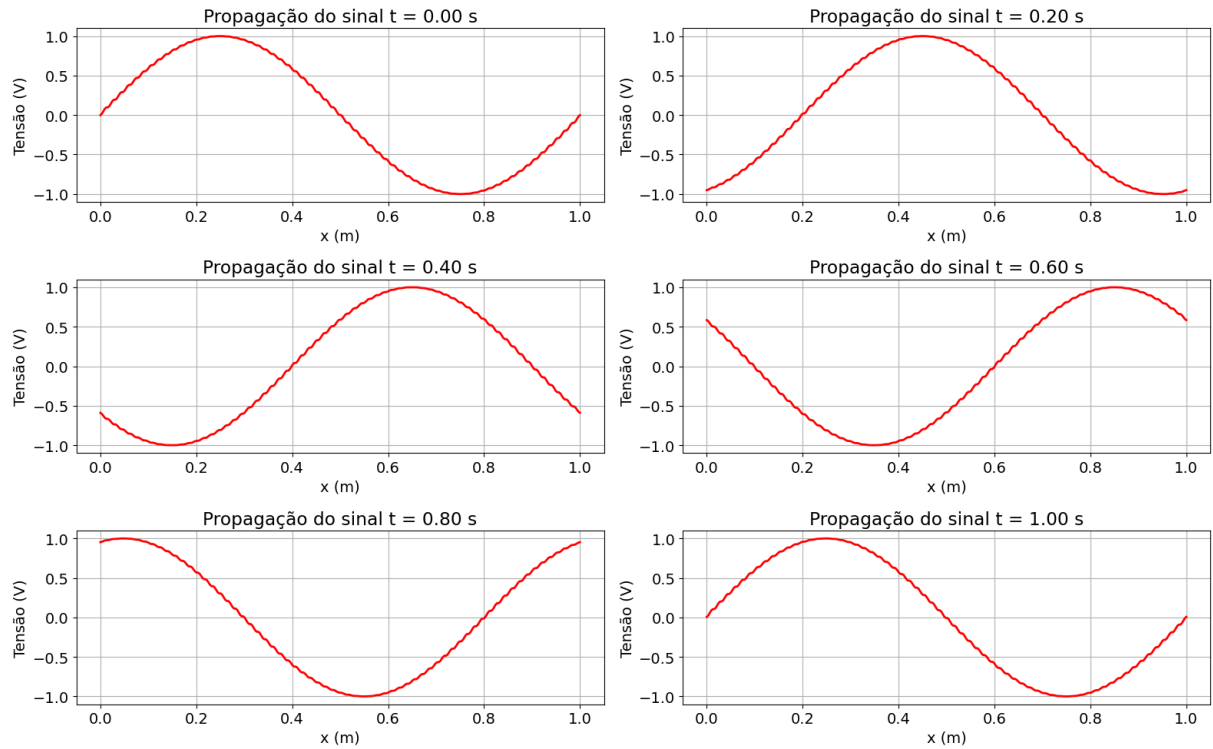


Figura 2 – Posição do pico do sinal ao longo do tempo para ambos os fluxos.

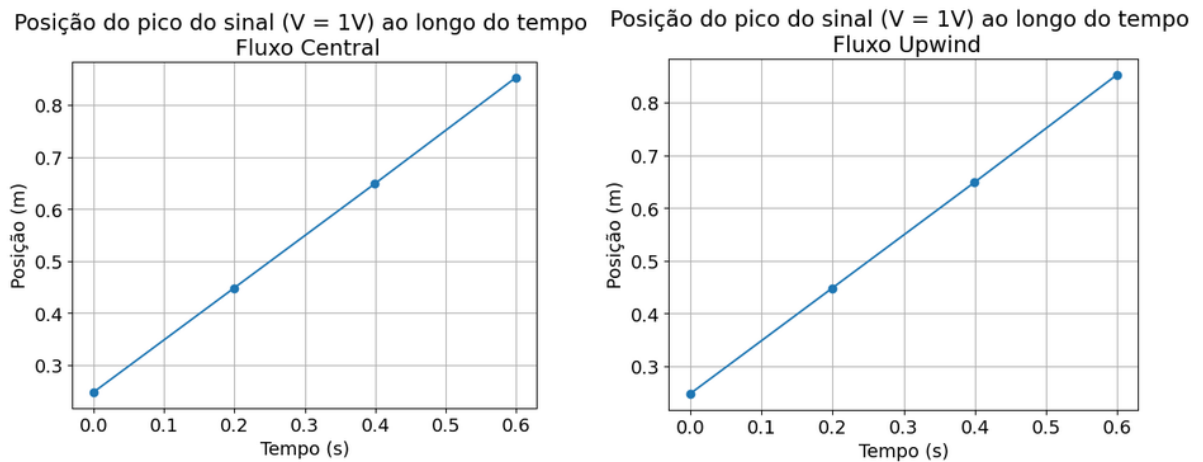
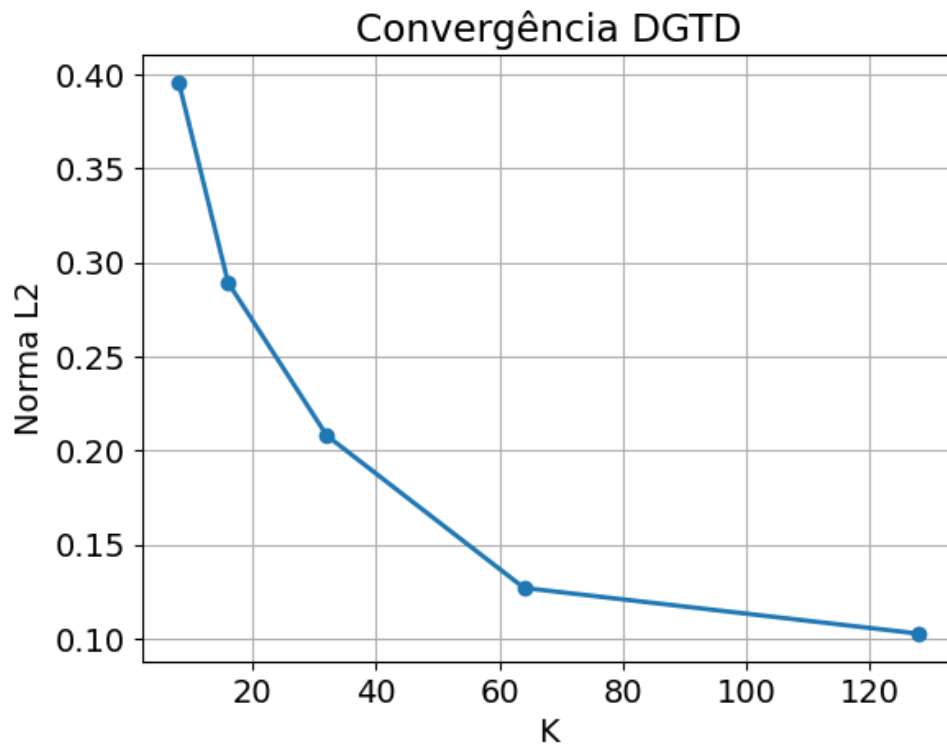


Figura 3 – Convergência da norma-L2 conforme aumenta-se o número de elementos K .



Era esperado que a norma se reduzisse por um fator mínimo de 2 a cada vez que K dobra. No entanto, isso não é observada na Figura 3, indicando que há possibilidades de melhorar o código.

1.7 Exercício 7

Usando a mesma discretização do exercício 6, mudando apenas a condição inicial para um pulso gaussiano no centro do domínio, obtemos a propagação ao longo do tempo exibida na Figura 4 para $a = +1$ m/s. Invertendo o sinal da velocidade, temos que o sinal se propaga para a esquerda com $a = -1$ m/s conforme mostra a Figura 5.

Figura 4 – Propagação do pulso gaussiano para $a = +1$ m/s.

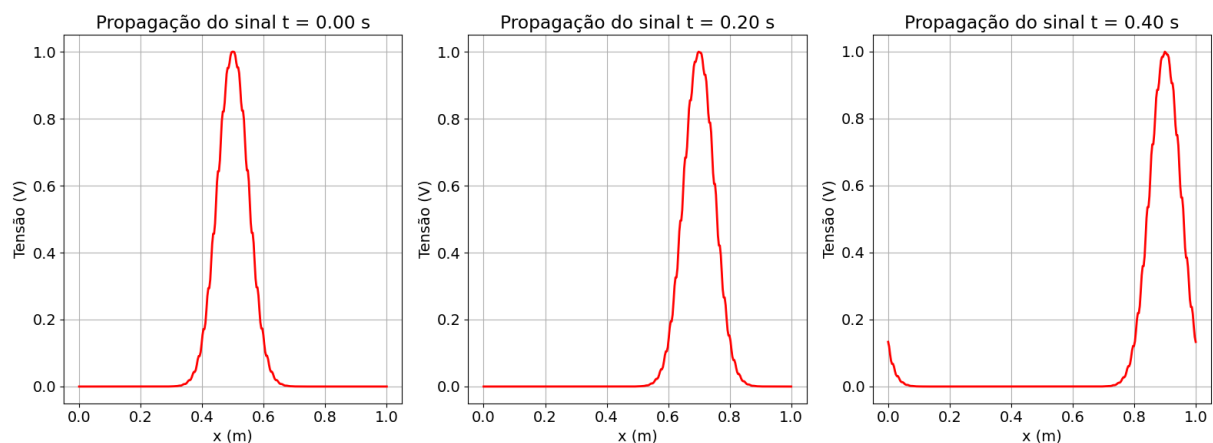
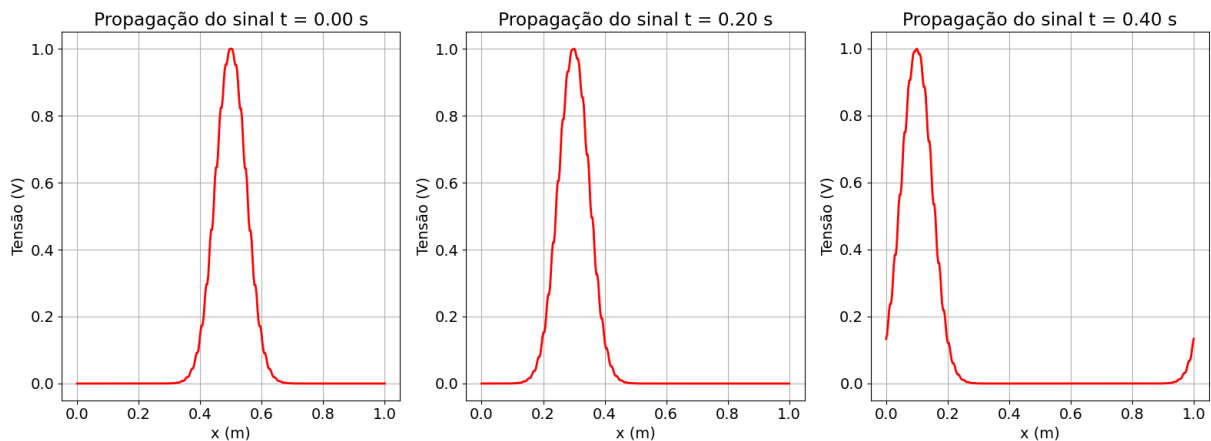
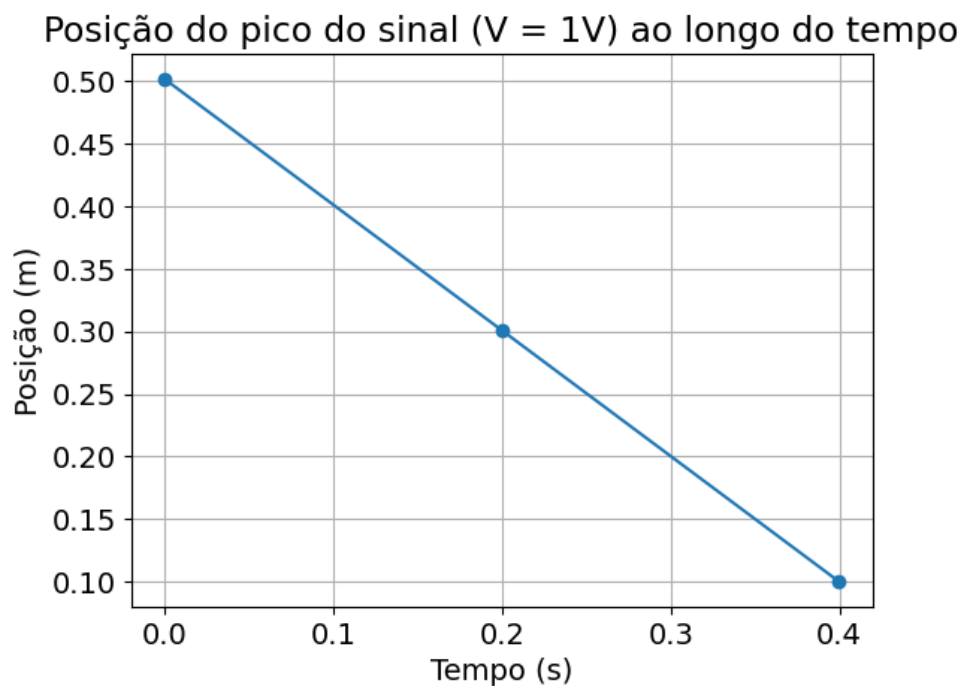


Figura 5 – Propagação do pulso gaussiano para $a = -1$ m/s.



Usando a mesma técnica do exercício 6 de monitorar a posição do pico do sinal ao longo do tempo e plotar a curva posição x tempo, tomando a regressão linear para calcular a velocidade, obtemos a velocidade numérica (para o pulso se deslocando para a esquerda) de $a_{NUM} = -1.004$ m/s, com um erro de 0.4 % em relação ao valor teórico, como mostra a Figura 6.

Figura 6 – Posição do pico do sinal ao longo do tempo.



1.8 Exercício 8

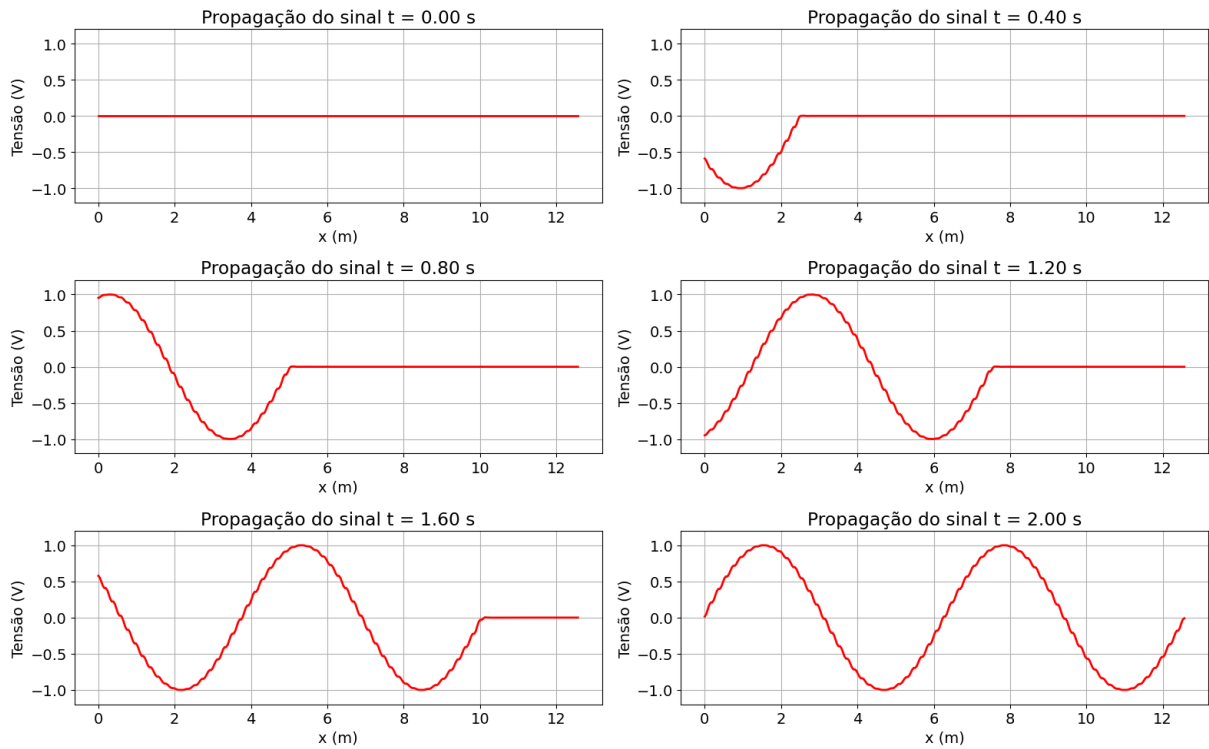
Todo o código usado nesse exercício encontra-se no Anexo B.

Usando o método do DGTD para a equação da advecção 1D e os seguintes parâmetros de discretização:

- $K = 64$
- $N = 4$
- $\Delta t = 2.5 \cdot 10^{-4} \text{ s}$ (CFL = 0.2)
- $\Delta x = 1.96 \cdot 10^{-1} \text{ m}$
- $T = 10 \text{ s} \implies N_t = 40000$

obtemos a seguinte propagação da onda para a direita a partir da condição inicial, ao aplicar a fonte senoidal na extremidade esquerda, exibida na Figura 7.

Figura 7 – Propagação da onda para a direita ao longo do tempo.



A solução analítica é dada por

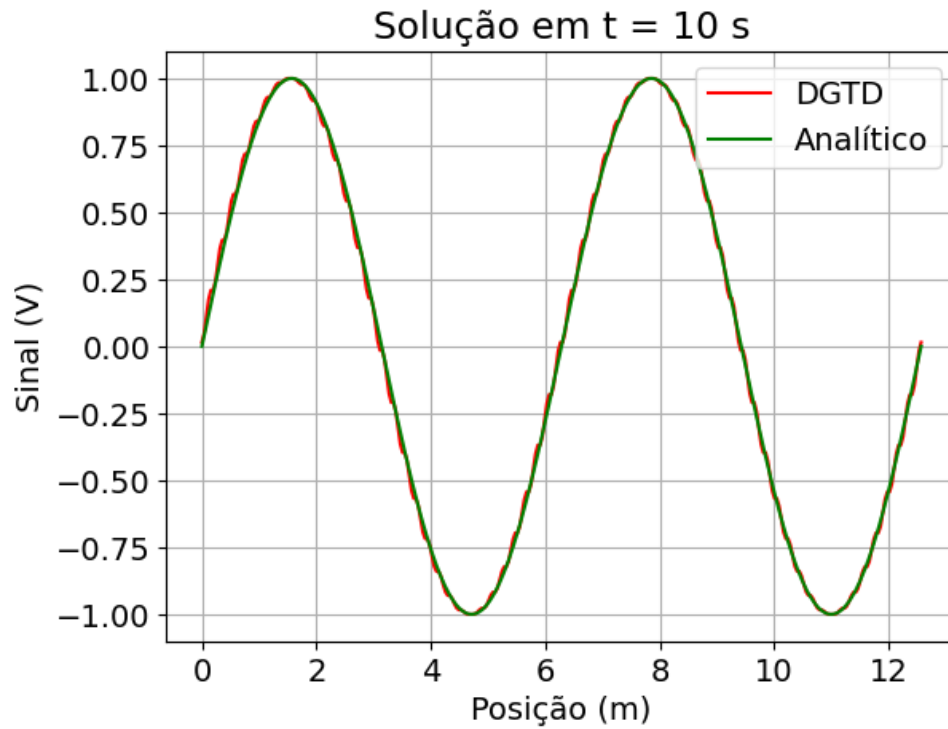
$$u_{exact}(x, t = 10 \text{ s}) = \sin(x)$$

que está sobreposta à solução numérica obtida como mostra a Figura 8.

A norma- L^2 calculada entre as soluções numérica e analítica, para diferentes valores de K , está exibida na Tabela 1. Vemos que o erro se reduz conforme aumentamos o número de elementos K da malha.

1.9 Exercício 9

O código usado nesse exercício é essencialmente o mesmo do exercício 6 presente no Anexo A, usando o seguinte comando para a condição inicial do pulso retangular:

Figura 8 – Solução em $t = 10$ s.Tabela 1 – Norma- L^2 em $t = 10$ s para diferentes valores de K .

K	Norma- L^2
8	0.904
16	0.673
32	0.485
64	0.34

```
u = np.where(np.abs(x_nodes - 0.3) <= (5 * dx)/2, 1.0, 0.0).flatten().copy()
```

O primeiro passo é definir a função degrau que será a condição inicial. A função usada está exibida na Figura 9, com largura de $5 \cdot \Delta x$.

Os parâmetros de discretização do domínio $[0, 1]$ adotados são:

- $K = 50$
- $N = 4$
- $\Delta t = 8 \cdot 10^{-5}$ s (CFL = 0.2)
- $\Delta x = 2 \cdot 10^{-2}$ m
- $T = 3$ s $\implies N_t = 37500$
- $a = 2$ m/s

Figura 9 – Condição inicial.

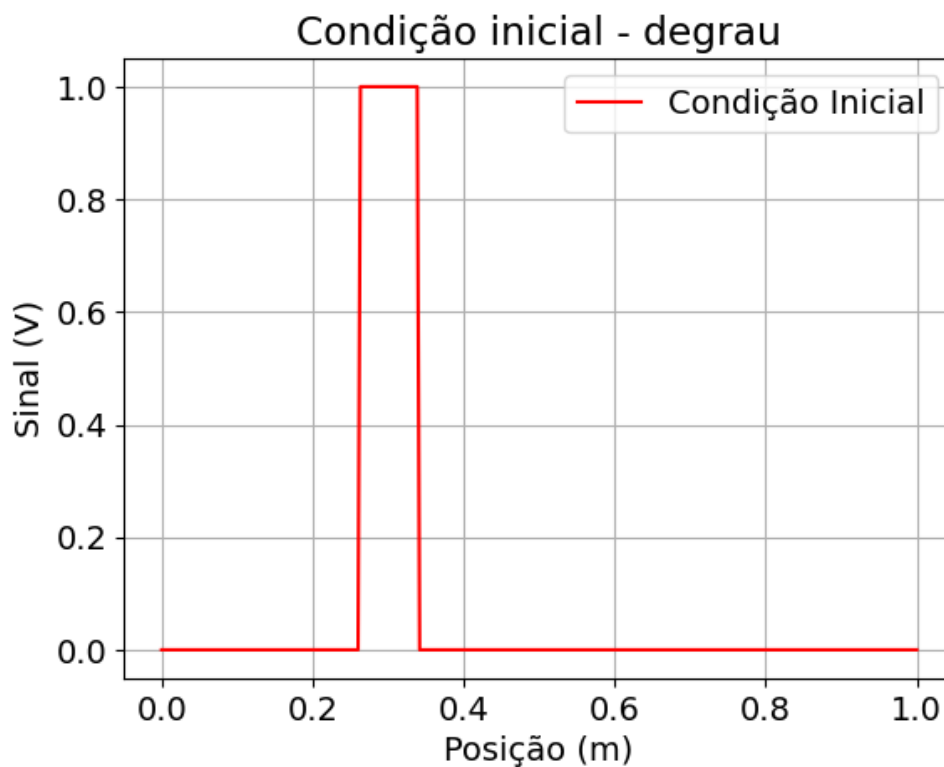
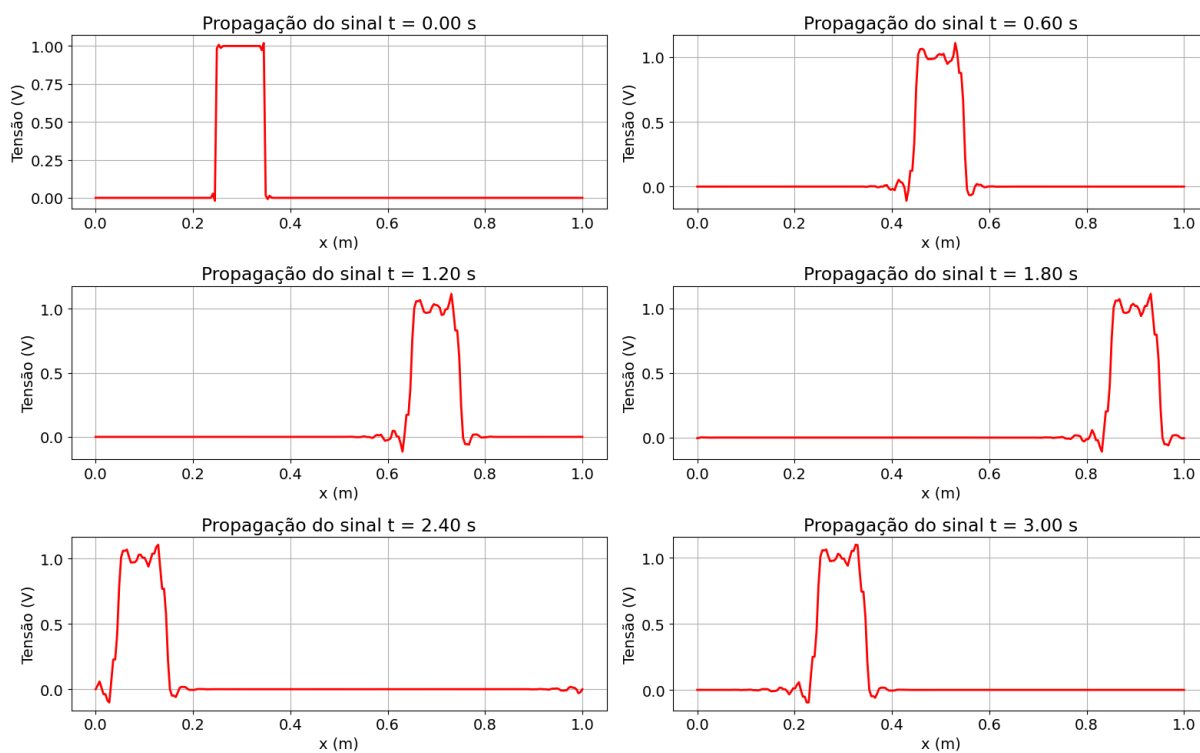


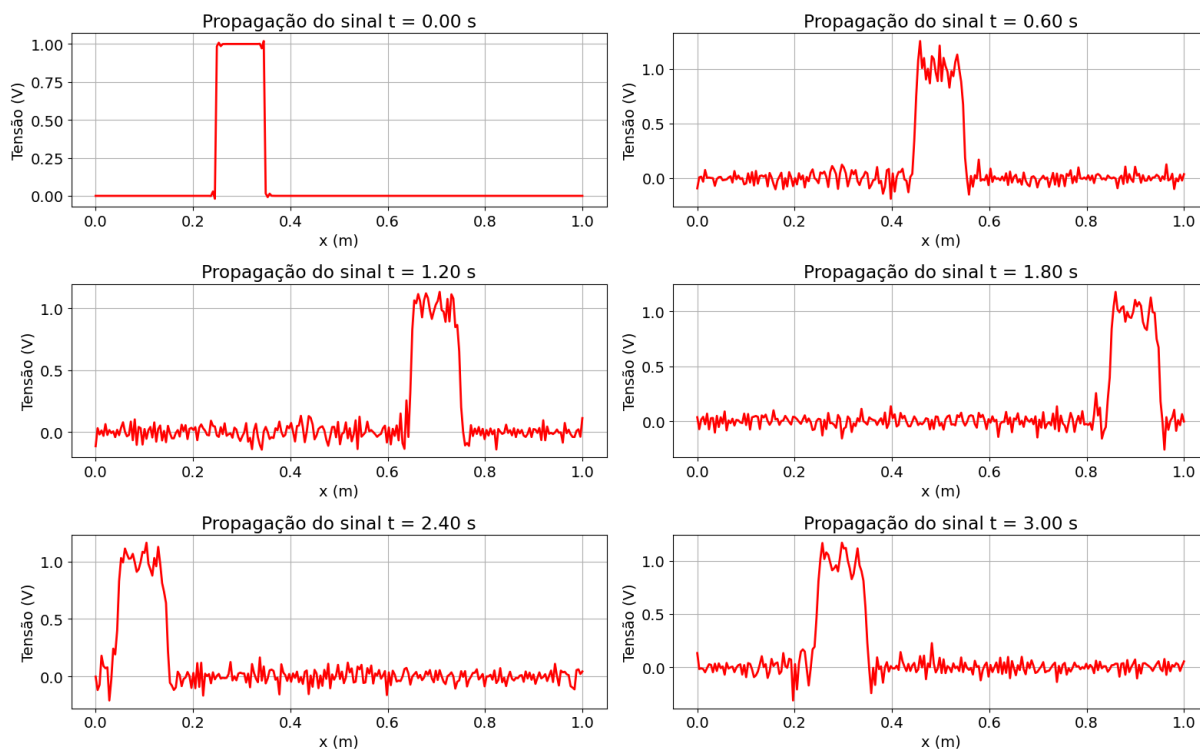
Figura 10 – Propagação do pulso retangular ao longo do tempo - fluxo upwind.



O resultado obtido encontra-se na Figura 10 para o fluxo numérico upwind.

Com o fluxo central, temos um resultado diferente como mostra a Figura 11. As descontinuidades introduzem oscilações ao longo de todo o sinal, ao contrário do fluxo central em que essas oscilações ocorrem apenas ao redor das bordas do pulso.

Figura 11 – Propagação do pulso retangular ao longo do tempo - fluxo central.



2 PARTE 2

2.1 Exercício 1

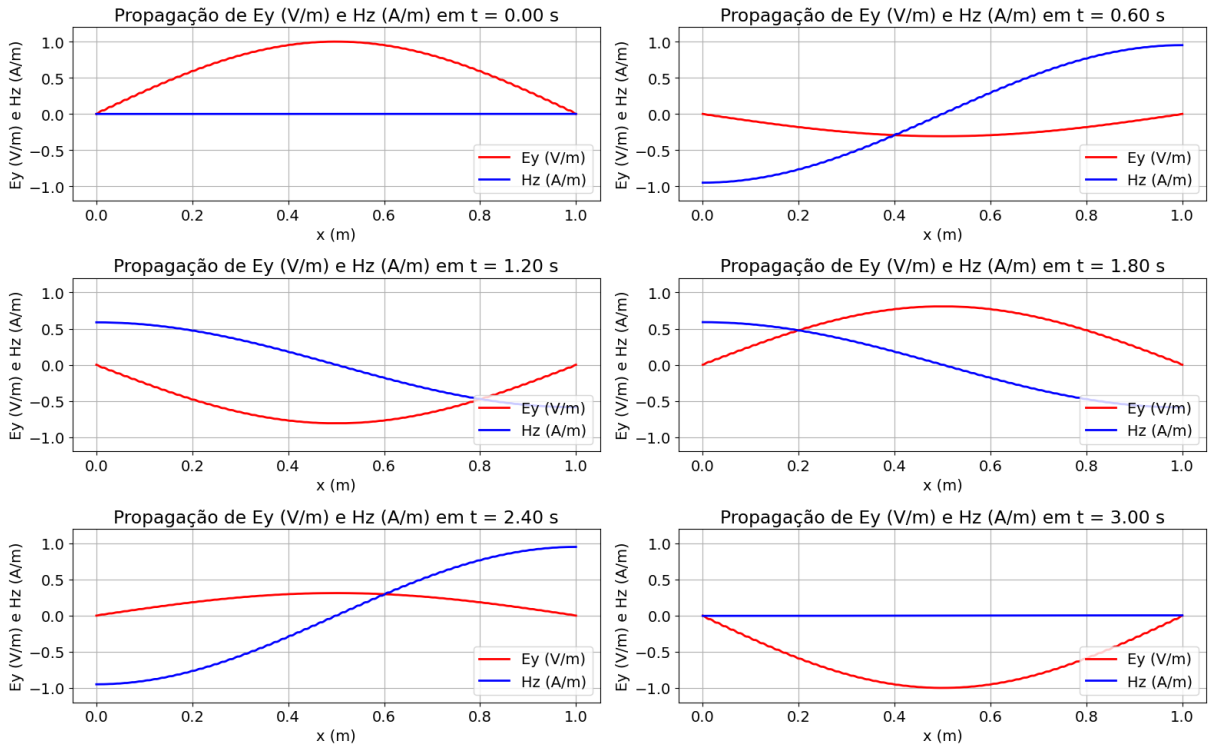
Todo o código usado nesse exercício encontra-se no Anexo C.

Usando o método do DGTD para as equações de Maxwell 1D com as seguintes discretizações:

- $K = 64$
- $N = 4$
- $\Delta t = 1.25 \cdot 10^{-4} \text{ s}$ (CFL = 0.2)
- $\Delta x = 1.56 \cdot 10^{-2} \text{ m}$
- $T = 3 \text{ s} \implies N_t = 24000$

obtendo o resultado exibido na Figura 12 para o fluxo central, e a Figura 13 para o fluxo upwind. Vemos que o resultado obtido é o mesmo para ambos os casos.

Figura 12 – Propagação de E_y e H_z ao longo do tempo - fluxo central.

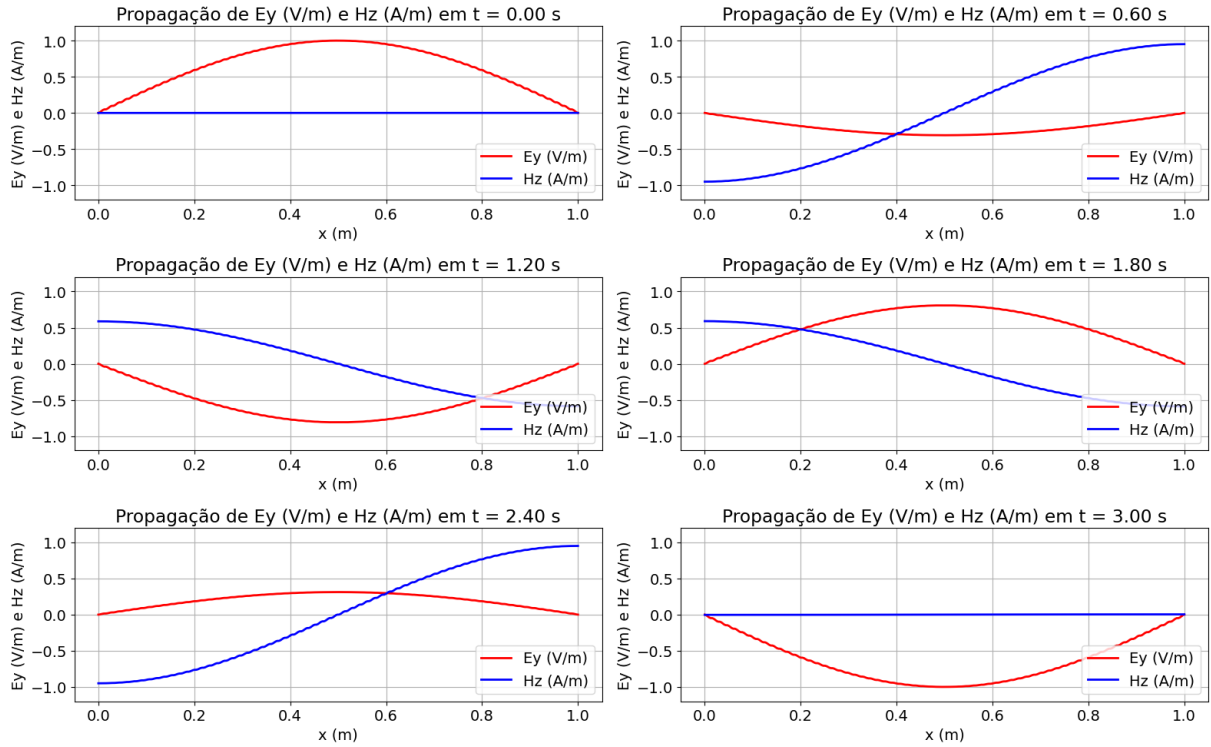


Sabemos de exercícios anteriores que a solução exata para esse problema é dada por

$$E_y = \sin(\pi x) \cos(\pi t) \quad , \quad H_z = -\cos(\pi x) \sin(\pi t)$$

de modo que podemos tomar a norma- L^2 para verificar o desempenho do método, como mostra a Tabela 2. Conforme esperado, o erro se reduz conforme aumentamos o

Figura 13 – Propagação de E_y e H_z ao longo do tempo - fluxo upwind.



número de elementos K da malha, contudo ainda não na velocidade esperada. Como o método de ordem elevada, era esperado que a norma- L^2 do erro se reduzisse no mínimo por um fator 4 quando aumentamos K por um fator 2.

Tabela 2 – Norma- L^2 para diferentes valores de K .

K	Norma- L^2
16	0.121
32	0.063
64	0.042

2.2 Exercício 2

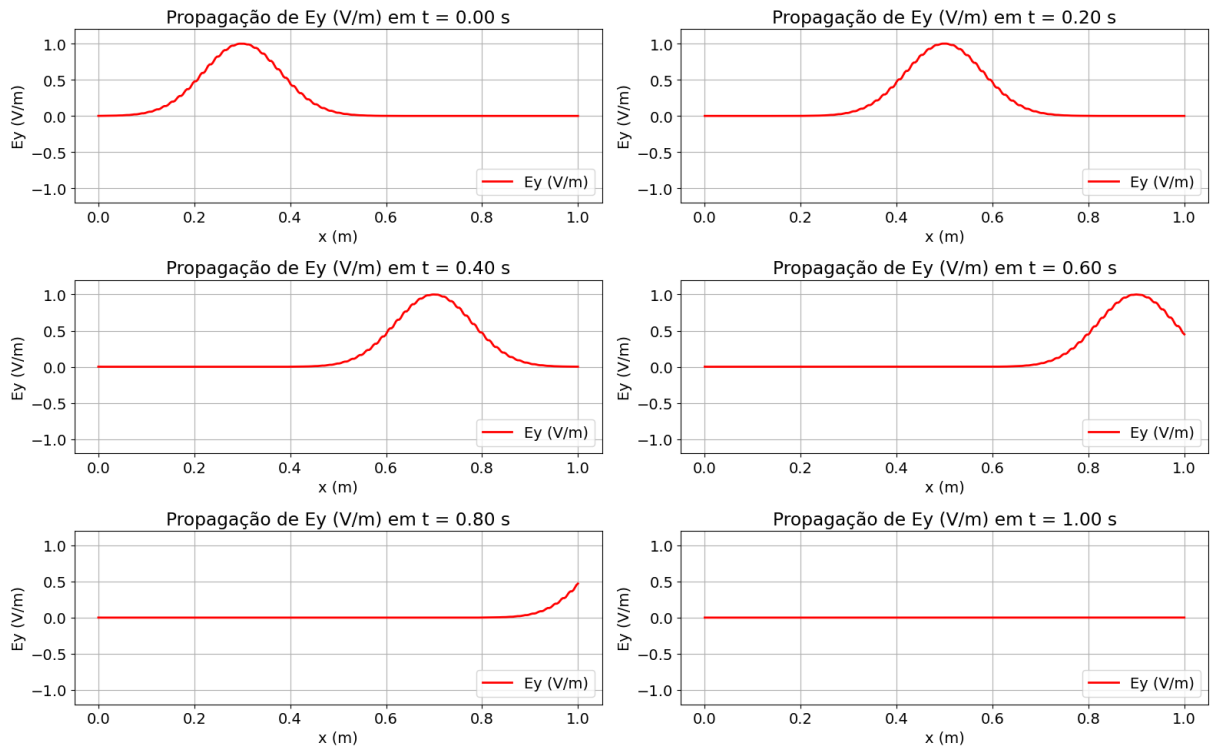
Todo o código usado nesse exercício encontra-se no Anexo D.

Com a mesma discretização do item anterior, agora propagamos um pulso gaussiano para a direita a fim de verificar a implementação da condição de contorno absorvente de SMC. O pulso inicial usado é dado por

$$E_y(x) = \exp \left[-\frac{1}{2} \frac{(x - 0.3)^2}{0.08} \right]$$

o que resulta na propagação exibida na Figura 14 com o fluxo central. Observa-se que não há nenhuma onda refletida a partir da borda direita após a passagem do sinal, indicando um bom funcionamento da condição de contorno.

Figura 14 – Propagação do pulso gaussiano para a direita em função do tempo - fluxo central.

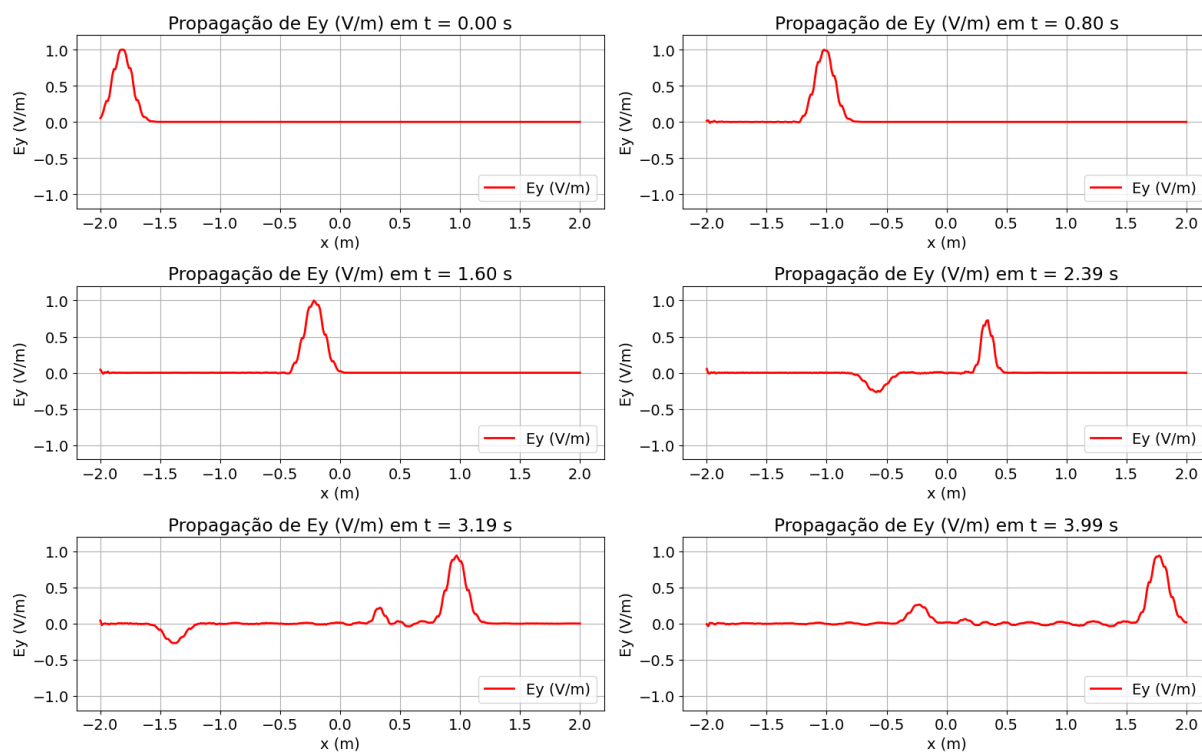


2.3 Exercício 3

Todo o código usado nesse exercício encontra-se no Anexo E.

A Figura 15 mostra a propagação do campo E_y com o pulso gaussiano para a direita com condições absorventes SMC e com a barra dielétrica $\epsilon = 3$ em $0 \leq x \leq 0.5$.

Observamos a presença de reflexões em $x = 0$ e $x = 0.5$, conforme esperado, uma vez que nesses pontos temos descontinuidades de impedância devido à mudanças de meio, resultando em reflexões no sinal.

Figura 15 – Propagação de E_y ao longo do tempo com a barra dielétrica.

ANEXO A – CÓDIGO EXERCÍCIO 1.6

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.special

# --- constantes ----
CENTRAL_FLUX = 0
UPWIND_FLUX = 1

A = np.array([0.0,
-567301805773.0 / 1357537059087.0,
-2404267990393.0 / 2016746695238.0,
-3550918686646.0 / 2091501179385.0,
-1275806237668.0 / 842570457699.0])

B = np.array([1432997174477.0 / 9575080441755.0,
5161836677717.0 / 13612068292357.0,
1720146321549.0 / 2090206949498.0,
3134564353537.0 / 4481467310338.0,
2277821191437.0 / 14882151754819.0])

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)

# --- domínio ---
vp = 1.0 * -1
kx = 2 * np.pi
xl, xr = 0, 1

# --- retorna matriz D - diferenciação ---
def get_D_matrix(nodes):
    N = len(nodes) - 1
    w = np.ones(N + 1)

    for j in range(N + 1):
        for i in range(N + 1):
            if i != j:
                w[j] *= 1.0 / (nodes[j] - nodes[i])
    D = np.zeros((N + 1, N + 1))

    for i in range(N + 1):
        for j in range(N + 1):
            if i != j:
                D[i, j] = (w[j] / w[i]) / (nodes[i] - nodes[j])
            D[i, i] = -np.sum(D[i, :])

    return D

# --- retorna fluxo numérico ---
def numerical_flux(uL, uR, a, flux_type):
    match flux_type:
        case 0:
            return 0.5 * a * (uL + uR)
        case 1:
            return a * uL if a > 0 else a * uR
        case _: # fallback central flux
            return 0.5 * a * (uL + uR)

```

```

# --- retorna pontos GLL de ordem polinomial N ---
def get_GLL_nodes(N):
    internal_nodes, _ = scipy.special.roots_jacobi(N - 1, 1, 1)
    r = np.concatenate([[ -1.0], internal_nodes, [ 1.0]]) # elemento de referência
    return r

# --- retorna pesos dos nós GLL de ordem polinomial N ---
def get_GLL_weights(nodes, N):
    N = len(nodes) - 1

    # Legendre polynomial of degree N
    P = scipy.special.legendre(N)

    # Evaluate P_N(x_i)
    PN = np.polyval(P, nodes)

    # Compute weights
    w = 2.0 / (N * (N + 1) * PN**2)

    return w

# --- operador DG ---
def build_dg_operator(K, N, a, flux_type):
    dx_elem = (xr - xl) / K
    xi = get_GLL_nodes(N)
    D_ref = get_D_matrix(xi)
    D_phys = (2.0 / dx_elem) * D_ref
    w = get_GLL_weights(xi, N)
    M_local = w * dx_elem / 2.0

    # Coordenadas físicas dos nós
    x_nodes = np.zeros((K, N + 1))
    for k in range(K):
        xL = xl + k * dx_elem
        x_nodes[k, :] = xL + 0.5 * (xi + 1) * dx_elem

    return {
        'K': K, 'N': N, 'dx_elem': dx_elem,
        'D_phys': D_phys, 'M_local': M_local,
        'x_nodes': x_nodes, 'a': a, 'flux_type': flux_type
    }

def rhs(u_flat, op):
    # puxa os dados do operador DG
    K, N = op['K'], op['N']
    D = op['D_phys']
    M = op['M_local']
    a = op['a']
    flux_type = op['flux_type']

    u = u_flat.reshape((K, N + 1))
    du = np.zeros_like(u)

    # itera sobre cada elemento e vai resolvendo em cada um
    for k in range(K):
        du[k, :] = -a * (D @ u[k, :])
        uL = u[k, N]
        uR = u[(k + 1) % K, 0]
        f_estrela_R = numerical_flux(uL, uR, a, flux_type)
        du[k, N] += a * (uL - f_estrela_R / a) / M[N]

```

```

    uL_prev = u[(k - 1) % K, N]
    uR_curr = u[k, 0]
    f_estrela_L = numerical_flux(uL_prev, uR_curr, a, flux_type)
    du[k, 0] += a * (-uR_curr + f_estrela_L / a) / M[0]

    return du.flatten()

# low storage Range Kutta quarta ordem
def lsrk4_step(u, rhs_func, dt, op):
    q = np.zeros_like(u)
    for i in range(5):
        q = A[i] * q + dt * rhs_func(u, op)
        u = u + B[i] * q

    return u

def DGTD(K, N, CFL, T_final, a, kx, flux_type):
    # gera o operador
    dg_operator = build_dg_operator(K, N, a, flux_type)
    x_nodes = dg_operator['x_nodes']

    # Condição inicial
    u = np.sin(kx * x_nodes).flatten().copy()
    u = np.exp(-0.5 * ((x_nodes - 0.5) / 0.05)**2).flatten().copy()

    # discretização
    dx = dg_operator['dx_elem']
    dt = CFL * dx / (np.abs(a) * (N + 1) ** 2)
    Nt = max(1, int(T_final / dt))
    dt = T_final / Nt # evita erro de arredondamento

    # snapshots
    u_history = []
    snapshots_list = np.linspace(0, Nt - 10, 6)

    print("INICIANDO MARCHA NO TEMPO")
    print(f"dx={dx:.3e}, dt={dt:.3e}, Nt={Nt}")

    # marcha no tempo
    for step in range(Nt):
        u = lsrk4_step(u, rhs, dt, dg_operator)

        if step in snapshots_list:
            u_history.append(u)

    return u_history, dx, dt, snapshots_list

# --- Parâmetros de simulação ---
K = 64 # numero de elementos
N = 4 # ordem polinomial dentro dos elementos
CFL = 0.2
T_final = 1

u_history, dx, dt, snapshots_list = DGTD(K, N, CFL, T_final, vp, kx, UPWIND_FLUX)

```

ANEXO B – CÓDIGO EXERCÍCIO 1.8

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.special

# --- constantes ----
CENTRAL_FLUX = 0
UPWIND_FLUX = 1

A = np.array([0.0,
-567301805773.0 / 1357537059087.0,
-2404267990393.0 / 2016746695238.0,
-3550918686646.0 / 2091501179385.0,
-1275806237668.0 / 842570457699.0])

B = np.array([1432997174477.0 / 9575080441755.0,
5161836677717.0 / 13612068292357.0,
1720146321549.0 / 2090206949498.0,
3134564353537.0 / 4481467310338.0,
2277821191437.0 / 14882151754819.0])

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)

# --- domínio ---
vp = 2 * np.pi
kx = 2 * np.pi
xl, xr = 0, 4 * np.pi

# --- retorna matriz D - diferenciação ---
def get_D_matrix(nodes):
    N = len(nodes) - 1
    w = np.ones(N + 1)

    for j in range(N + 1):
        for i in range(N + 1):
            if i != j:
                w[j] *= 1.0 / (nodes[j] - nodes[i])
    D = np.zeros((N + 1, N + 1))

    for i in range(N + 1):
        for j in range(N + 1):
            if i != j:
                D[i, j] = (w[j] / w[i]) / (nodes[i] - nodes[j])
            D[i, i] = -np.sum(D[i, :])

    return D

# --- retorna fluxo numérico ---
def numerical_flux(uL, uR, a, flux_type):
    match flux_type:
        case 0:
            return 0.5 * a * (uL + uR)
        case 1:
            return a * uL if a > 0 else a * uR
        case _: # fallback central flux
            return 0.5 * a * (uL + uR)

```

```

# --- retorna pontos GLL de ordem polinomial N ---
def get_GLL_nodes(N):
    internal_nodes, _ = scipy.special.roots_jacobi(N - 1, 1, 1)
    r = np.concatenate([[ -1.0], internal_nodes, [ 1.0]]) # elemento de referência
    return r

# --- retorna pesos dos nós GLL de ordem polinomial N ---
def get_GLL_weights(nodes, N):
    N = len(nodes) - 1

    # Legendre polynomial of degree N
    P = scipy.special.legendre(N)

    # Evaluate  $P_N(x_i)$ 
    PN = np.polyval(P, nodes)

    # Compute weights
    w = 2.0 / (N * (N + 1) * PN**2)

    return w

# --- operador DG ---
def build_dg_operator(K, N, a, flux_type):
    dx_elem = (xr - xl) / K
    xi = get_GLL_nodes(N)
    D_ref = get_D_matrix(xi)
    D_phys = (2.0 / dx_elem) * D_ref
    w = get_GLL_weights(xi, N)
    M_local = w * dx_elem / 2.0

    # Coordenadas físicas dos nós
    x_nodes = np.zeros((K, N + 1))
    for k in range(K):
        xL = xl + k * dx_elem
        x_nodes[k, :] = xL + 0.5 * (xi + 1) * dx_elem

    return {
        'K': K, 'N': N, 'dx_elem': dx_elem,
        'D_phys': D_phys, 'M_local': M_local,
        'x_nodes': x_nodes, 'a': a, 'flux_type': flux_type
    }

def rhs(u_flat, op, step, dt):
    # puxa os dados do operador DG
    K, N = op['K'], op['N']
    D = op['D_phys']
    M = op['M_local']
    a = op['a']
    flux_type = op['flux_type']

    u = u_flat.reshape((K, N + 1))
    du = np.zeros_like(u)

    # itera sobre cada elemento e vai resolvendo em cada um
    for k in range(K):
        du[k, :] = -a * (D @ u[k, :])

        # fronteira direita
        if k == K - 1:
            uL = u[k, N]

```

```

        uR = uL
    else:
        uL = u[k, N]
        uR = u[k+1, 0]
        f_estrela_R = numerical_flux(uL, uR, a, flux_type)
        du[k, N] += a * (uL - f_estrela_R / a) / M[N]

    # fronteira esquerda
    if k == 0:
        uL = -np.sin(2.0 * np.pi * step * dt) # fonte
        uR = u[k, 0]
    else:
        uL = u[k-1, N]
        uR = u[k, 0]
        f_estrela_L = numerical_flux(uL, uR, a, flux_type)
        du[k, 0] += a * (-uR + f_estrela_L / a) / M[0]

    return du.flatten()

# low storage Range Kutta quarta ordem
def lsrk4_step(u, rhs_func, dt, op, step):
    q = np.zeros_like(u)
    for i in range(5):
        q = A[i] * q + dt * rhs_func(u, op, step, dt)
        u = u + B[i] * q

    return u

def DGTD(K, N, CFL, T_final, a, kx, flux_type):
    # gera o operador
    dg_operator = build_dg_operator(K, N, a, flux_type)
    x_nodes = dg_operator['x_nodes']

    # Condição inicial
    u = np.sin(kx * x_nodes).flatten().copy()
    u = np.exp(-0.5 * ((x_nodes - 0.5) / 0.05)**2).flatten().copy()
    u = np.zeros_like(x_nodes.flatten())

    # discretização
    dx = dg_operator['dx_elem']
    dt = CFL * dx / (np.abs(a) * (N + 1) ** 2)
    Nt = max(1, int(T_final / dt))
    dt = T_final / Nt # evita erro de arredondamento

    # snapshots
    u_history = []
    snapshots_list = np.linspace(0, Nt - 10, 6)

    print("INICIANDO_MARCHA_NO_TEMPO")
    print(f"dx={dx:.3e}, dt={dt:.3e}, Nt={Nt}")

    # marcha no tempo
    for step in range(Nt):
        u = lsrk4_step(u, rhs, dt, dg_operator, step)

        if step in snapshots_list:
            u_history.append(u)

    return u_history, dx, dt, snapshots_list

```

```
# --- Parâmetros de simulação ----  
K = 32 # numero de elementos  
N = 4 # ordem polinomial dentro dos elementos  
CFL = 0.2  
T_final = 10  
  
u_history, dx, dt, snapshots_list = DGTD(K, N, CFL, T_final, vp, kx, UPWIND_FLUX)
```


ANEXO C – CÓDIGO EXERCÍCIO 2.1

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.special

# --- constantes ----
CENTRAL_FLUX = 0
UPWIND_FLUX = 1

A = np.array([0.0,
-567301805773.0 / 1357537059087.0,
-2404267990393.0 / 2016746695238.0,
-3550918686646.0 / 2091501179385.0,
-1275806237668.0 / 842570457699.0])

B = np.array([1432997174477.0 / 9575080441755.0,
5161836677717.0 / 13612068292357.0,
1720146321549.0 / 2090206949498.0,
3134564353537.0 / 4481467310338.0,
2277821191437.0 / 14882151754819.0])

# --- parâmetros físicos ---
eps = 1
mu = 1

# --- domínio ---
vp = 1 / np.sqrt(eps * mu)
Z = np.sqrt(mu / eps)
xl, xr = 0, 1

# --- retorna matriz D - diferenciação ---
def get_D_matrix(nodes):
    N = len(nodes) - 1
    w = np.ones(N + 1)

    for j in range(N + 1):
        for i in range(N + 1):
            if i != j:
                w[j] *= 1.0 / (nodes[j] - nodes[i])
    D = np.zeros((N + 1, N + 1))

    for i in range(N + 1):
        for j in range(N + 1):
            if i != j:
                D[i, j] = (w[j] / w[i]) / (nodes[i] - nodes[j])
            D[i, i] = -np.sum(D[i, :])

    return D

# --- retorna fluxo numérico ---
def numerical_flux(uL, uR, a, flux_type):
    match flux_type:
        case 0:
            return 0.5 * a * (uL + uR)
        case 1:
            return a * uL if a > 0 else a * uR
        case _: # fallback central flux
            return 0.5 * a * (uL + uR)

```

```

# --- retorna pontos GLL de ordem polinomial N ---
def get_GLL_nodes(N):
    internal_nodes, _ = scipy.special.roots_jacobi(N - 1, 1, 1)
    r = np.concatenate([[-1.0], internal_nodes, [1.0]]) # elemento de referência
    return r

# --- retorna pesos dos nós GLL de ordem polinomial N ---
def get_GLL_weights(nodes, N):
    N = len(nodes) - 1

    # Legendre polynomial of degree N
    P = scipy.special.legendre(N)

    # Evaluate P_N(x_i)
    PN = np.polyval(P, nodes)

    # Compute weights
    w = 2.0 / (N * (N + 1) * PN**2)

    return w

# --- operador DG ---
def build_dg_operator(K, N, a, flux_type):
    dx_elem = (xr - xl) / K
    xi = get_GLL_nodes(N)
    D_ref = get_D_matrix(xi)
    D_phys = (2.0 / dx_elem) * D_ref
    w = get_GLL_weights(xi, N)
    M_local = w * dx_elem / 2.0

    # Coordenadas físicas dos nós
    x_nodes = np.zeros((K, N + 1))
    for k in range(K):
        xL = xl + k * dx_elem
        x_nodes[k, :] = xL + 0.5 * (xi + 1) * dx_elem

    return {
        'K': K, 'N': N, 'dx_elem': dx_elem,
        'D_phys': D_phys, 'M_local': M_local,
        'x_nodes': x_nodes, 'a': a, 'flux_type': flux_type
    }

def rhs(u_flat, op):
    # puxa os dados do operador DG
    K, N = op['K'], op['N']
    D = op['D_phys']
    M = op['M_local']
    a = op['a']
    flux_type = op['flux_type']

    u = u_flat.reshape((K, N + 1, 2))
    du = np.zeros_like(u)

    # itera sobre cada elemento e vai resolvendo em cada um
    for k in range(K):
        du[k, :, 0] = - (D @ u[k, :, 1]) / eps
        du[k, :, 1] = - (D @ u[k, :, 0]) / mu

        if k == K-1:
            Ey_int = u[k, N, 0]

```

```

    Hz_int = u[k, N, 1]
    Ey_star = 0.0
    Hz_star = Hz_int
else:
    Ey_L = u[k, N, 0]; Hz_L = u[k, N, 1]
    Ey_R = u[k+1, 0, 0]; Hz_R = u[k+1, 0, 1]
    if flux_type == 'central':
        Ey_star = 0.5*(Ey_L + Ey_R)
        Hz_star = 0.5*(Hz_L + Hz_R)
    else: # upwind
        Ey_star = 0.5*(Ey_L + Ey_R) + 0.5*Z*(Hz_L - Hz_R)
        Hz_star = 0.5*(Hz_L + Hz_R) + 0.5/Z*(Ey_L - Ey_R)

# Correção no último nó (N): sinal negativo
du[k, N, 0] += - (Hz_star - u[k, N, 1]) / M[N]
du[k, N, 1] += - (Ey_star - u[k, N, 0]) / M[N]

# --- Face esquerda (x_L) ---
if k == 0:
    # PEC em x=0
    Ey_int = u[k, 0, 0]
    Hz_int = u[k, 0, 1]
    Ey_star = 0.0
    Hz_star = Hz_int
else:
    Ey_L = u[k-1, N, 0]; Hz_L = u[k-1, N, 1]
    Ey_R = u[k, 0, 0]; Hz_R = u[k, 0, 1]
    if flux_type == 'central':
        Ey_star = 0.5*(Ey_L + Ey_R)
        Hz_star = 0.5*(Hz_L + Hz_R)
    else:
        Ey_star = 0.5*(Ey_L + Ey_R) + 0.5*Z*(Hz_L - Hz_R)
        Hz_star = 0.5*(Hz_L + Hz_R) + 0.5/Z*(Ey_L - Ey_R)

# Correção no primeiro nó (0): sinal positivo
du[k, 0, 0] += (Hz_star - u[k, 0, 1]) / M[0]
du[k, 0, 1] += (Ey_star - u[k, 0, 0]) / M[0]

return du.flatten()

# low storage Range Kutta quarta ordem
def lsrk4_step(u, rhs_func, dt, op):
    q = np.zeros_like(u)
    for i in range(5):
        q = A[i] * q + dt * rhs_func(u, op)
        u = u + B[i] * q

    return u

def DGTD(K, N, CFL, T_final, a, flux_type):
    # gera o operador
    dg_operator = build_dg_operator(K, N, a, flux_type)
    x_nodes = dg_operator['x_nodes']

    # discretização
    dx = dg_operator['dx_elem']
    dt = CFL * dx / (np.abs(a) * (N + 1) ** 2)
    Nt = max(1, int(T_final / dt))
    dt = T_final / Nt # evita erro de arredondamento

```

```

# Condição inicial
u = np.stack([np.sin(np.pi * x_nodes), np.zeros_like(x_nodes)], axis=-1).flatten().
    copy()

# snapshots
u_history = []
snapshots_list = np.linspace(0, Nt - 10, 6)

print("INICIANDO_MARCHA_NO_TEMPO")
print(f"dx={dx:.3e}, dt={dt:.3e}, Nt={Nt}")

# marcha no tempo
for step in range(Nt):
    u = lsrk4_step(u, rhs, dt, dg_operator)

    if step in snapshots_list:
        u_history.append(u.reshape((K, N+1, 2)))

return u_history, dx, dt, snapshots_list

# --- Parâmetros de simulação ----
K = 64 # numero de elementos
N = 4 # ordem polinomial dentro dos elementos
CFL = 0.2
T_final = 3

u_history, dx, dt, snapshots_list = DGTD(K, N, CFL, T_final, vp, UPWIND_FLUX)

```

ANEXO D – CÓDIGO EXERCÍCIO 2.2

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.special

# --- constantes ----
CENTRAL_FLUX = 0
UPWIND_FLUX = 1

A = np.array([0.0,
-567301805773.0 / 1357537059087.0,
-2404267990393.0 / 2016746695238.0,
-3550918686646.0 / 2091501179385.0,
-1275806237668.0 / 842570457699.0])

B = np.array([1432997174477.0 / 9575080441755.0,
5161836677717.0 / 13612068292357.0,
1720146321549.0 / 2090206949498.0,
3134564353537.0 / 4481467310338.0,
2277821191437.0 / 14882151754819.0])

# --- parâmetros físicos ---
eps = 1
mu = 1

# --- domínio ---
vp = 1 / np.sqrt(eps * mu)
Z = np.sqrt(mu / eps)
xl, xr = 0, 1

# --- retorna matriz D - diferenciação ---
def get_D_matrix(nodes):
    N = len(nodes) - 1
    w = np.ones(N + 1)

    for j in range(N + 1):
        for i in range(N + 1):
            if i != j:
                w[j] *= 1.0 / (nodes[j] - nodes[i])
    D = np.zeros((N + 1, N + 1))

    for i in range(N + 1):
        for j in range(N + 1):
            if i != j:
                D[i, j] = (w[j] / w[i]) / (nodes[i] - nodes[j])
            D[i, i] = -np.sum(D[i, :])

    return D

# --- retorna fluxo numérico ---
def numerical_flux(uL, uR, a, flux_type):
    match flux_type:
        case 0:
            return 0.5 * a * (uL + uR)
        case 1:
            return a * uL if a > 0 else a * uR
        case _: # fallback central flux
            return 0.5 * a * (uL + uR)

```

```

# --- retorna pontos GLL de ordem polinomial N ---
def get_GLL_nodes(N):
    internal_nodes, _ = scipy.special.roots_jacobi(N - 1, 1, 1)
    r = np.concatenate([[ -1.0], internal_nodes, [1.0]]) # elemento de referência
    return r

# --- retorna pesos dos nós GLL de ordem polinomial N ---
def get_GLL_weights(nodes, N):
    N = len(nodes) - 1

    # Legendre polynomial of degree N
    P = scipy.special.legendre(N)

    # Evaluate  $P_N(x_i)$ 
    PN = np.polyval(P, nodes)

    # Compute weights
    w = 2.0 / (N * (N + 1) * PN**2)

    return w

# --- operador DG ---
def build_dg_operator(K, N, a, flux_type):
    dx_elem = (xr - xl) / K
    xi = get_GLL_nodes(N)
    D_ref = get_D_matrix(xi)
    D_phys = (2.0 / dx_elem) * D_ref
    w = get_GLL_weights(xi, N)
    M_local = w * dx_elem / 2.0

    # Coordenadas físicas dos nós
    x_nodes = np.zeros((K, N + 1))
    for k in range(K):
        xL = xl + k * dx_elem
        x_nodes[k, :] = xL + 0.5 * (xi + 1) * dx_elem

    return {
        'K': K, 'N': N, 'dx_elem': dx_elem,
        'D_phys': D_phys, 'M_local': M_local,
        'x_nodes': x_nodes, 'a': a, 'flux_type': flux_type
    }

def rhs(u_flat, op):
    # puxa os dados do operador DG
    K, N = op['K'], op['N']
    D = op['D_phys']
    M = op['M_local']
    a = op['a']
    flux_type = op['flux_type']

    u = u_flat.reshape((K, N + 1, 2))
    du = np.zeros_like(u)

    # itera sobre cada elemento e vai resolvendo em cada um
    for k in range(K):
        du[k, :, 0] = - (D @ u[k, :, 1]) / eps
        du[k, :, 1] = - (D @ u[k, :, 0]) / mu

    # --- Face direita ( $x_R$ ) ---
    if k == K-1:

```

```

    Ey_int = u[k, N, 0]
    Hz_int = u[k, N, 1]
    Ey_ext = 0.5 * (Ey_int + Z*Hz_int)
    Hz_ext = 0.5 * (Ey_int + Z*Hz_int) / Z
    EY_L = Ey_int
    HZ_L = Hz_int
    EY_R = Ey_ext
    HZ_R = Hz_ext
else:
    EY_L = u[k, N, 0]
    HZ_L = u[k, N, 1]
    EY_R = u[k+1, 0, 0]
    HZ_R = u[k+1, 0, 1]
if flux_type == 'central':
    EY_star = 0.5*(EY_L + EY_R)
    HZ_star = 0.5*(HZ_L + HZ_R)
else:
    EY_star = 0.5*(EY_L + EY_R) + 0.5*Z*(HZ_L - HZ_R)
    HZ_star = 0.5*(HZ_L + HZ_R) + 0.5/Z*(EY_L - EY_R)

du[k, N, 0] += - (HZ_star - HZ_L) / M[N]
du[k, N, 1] += - (EY_star - EY_L) / M[N]

if k == 0:
    Ey_int = u[k, 0, 0]
    Hz_int = u[k, 0, 1]
    Ey_ext = 0.5 * (Ey_int - Z*Hz_int)
    Hz_ext = -0.5 * (Ey_int - Z*Hz_int) / Z
    EY_L = Ey_ext
    HZ_L = Hz_ext
    EY_R = Ey_int
    HZ_R = Hz_int
else:
    EY_L = u[k-1, N, 0]
    HZ_L = u[k-1, N, 1]
    EY_R = u[k, 0, 0]
    HZ_R = u[k, 0, 1]
if flux_type == 'central':
    EY_star = 0.5*(EY_L + EY_R)
    HZ_star = 0.5*(HZ_L + HZ_R)
else:
    EY_star = 0.5*(EY_L + EY_R) + 0.5*Z*(HZ_L - HZ_R)
    HZ_star = 0.5*(HZ_L + HZ_R) + 0.5/Z*(EY_L - EY_R)

# Correção no primeiro nó (sinal positivo)
du[k, 0, 0] += (HZ_star - HZ_R) / M[0]
du[k, 0, 1] += (EY_star - EY_R) / M[0]

return du.flatten()

# low storage Range Kutta quarta ordem
def lsrk4_step(u, rhs_func, dt, op):
    q = np.zeros_like(u)
    for i in range(5):
        q = A[i] * q + dt * rhs_func(u, op)
        u = u + B[i] * q

    return u

```

```

def DGTD(K, N, CFL, T_final, a, flux_type):
    # gera o operador
    dg_operator = build_dg_operator(K, N, a, flux_type)
    x_nodes = dg_operator['x_nodes']

    # discretização
    dx = dg_operator['dx_elem']
    dt = CFL * dx / (np.abs(a) * (N + 1) ** 2)
    Nt = max(1, int(T_final / dt))
    dt = T_final / Nt # evita erro de arredondamento

    # Condição inicial
    Ey_0 = np.exp(-0.5 * ((x_nodes - 0.3) / 0.08)**2)
    Hz_0 = Ey_0 / Z
    u = np.stack([Ey_0, Hz_0], axis=-1).flatten().copy()

    # snapshots
    u_history = []
    snapshots_list = np.linspace(0, Nt - 10, 6)

    print("INICIANDO MARCHA NO TEMPO")
    print(f"dx={dx:.3e}, dt={dt:.3e}, Nt={Nt}")

    # marcha no tempo
    for step in range(Nt):
        u = lsrk4_step(u, rhs, dt, dg_operator)

        if step in snapshots_list:
            u_history.append(u.reshape((K, N+1, 2)))

    return u_history, dx, dt, snapshots_list

# --- Parâmetros de simulação ---
K = 64 # numero de elementos
N = 4 # ordem polinomial dentro dos elementos
CFL = 0.4
T_final = 1

u_history, dx, dt, snapshots_list = DGTD(K, N, CFL, T_final, vp, 'central')

```


ANEXO E – CÓDIGO EXERCÍCIO 2.3

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.special

# --- constantes ----
CENTRAL_FLUX = 0
UPWIND_FLUX = 1

A = np.array([0.0,
-567301805773.0 / 1357537059087.0,
-2404267990393.0 / 2016746695238.0,
-3550918686646.0 / 2091501179385.0,
-1275806237668.0 / 842570457699.0])

B = np.array([1432997174477.0 / 9575080441755.0,
5161836677717.0 / 13612068292357.0,
1720146321549.0 / 2090206949498.0,
3134564353537.0 / 4481467310338.0,
2277821191437.0 / 14882151754819.0])

# --- parâmetros físicos ---
eps = 1
mu = 1

# --- domínio ---
vp = 1 / np.sqrt(eps * mu)
Z = np.sqrt(mu / eps)
xl, xr = -2, 2

# --- retorna matriz D - diferenciação ---
def get_D_matrix(nodes):
    N = len(nodes) - 1
    w = np.ones(N + 1)

    for j in range(N + 1):
        for i in range(N + 1):
            if i != j:
                w[j] *= 1.0 / (nodes[j] - nodes[i])
    D = np.zeros((N + 1, N + 1))

    for i in range(N + 1):
        for j in range(N + 1):
            if i != j:
                D[i, j] = (w[j] / w[i]) / (nodes[i] - nodes[j])
            D[i, i] = -np.sum(D[i, :])

    return D

# --- retorna fluxo numérico ---
def numerical_flux(uL, uR, a, flux_type):
    match flux_type:
        case 0:
            return 0.5 * a * (uL + uR)
        case 1:
            return a * uL if a > 0 else a * uR
        case _: # fallback central flux
            return 0.5 * a * (uL + uR)

```

```

# --- retorna pontos GLL de ordem polinomial N ---
def get_GLL_nodes(N):
    internal_nodes, _ = scipy.special.roots_jacobi(N - 1, 1, 1)
    r = np.concatenate([[ -1.0], internal_nodes, [1.0]]) # elemento de referência
    return r

# --- retorna pesos dos nós GLL de ordem polinomial N ---
def get_GLL_weights(nodes, N):
    N = len(nodes) - 1

    # Legendre polynomial of degree N
    P = scipy.special.legendre(N)

    # Evaluate P_N(x_i)
    PN = np.polyval(P, nodes)

    # Compute weights
    w = 2.0 / (N * (N + 1) * PN**2)

    return w

# --- operador DG ---
def build_dg_operator(K, N, a, flux_type):
    dx_elem = (xr - xl) / K
    xi = get_GLL_nodes(N)
    D_ref = get_D_matrix(xi)
    D_phys = (2.0 / dx_elem) * D_ref
    w = get_GLL_weights(xi, N)
    M_local = w * dx_elem / 2.0

    # Coordenadas físicas dos nós
    x_nodes = np.zeros((K, N + 1))
    for k in range(K):
        xL = xl + k * dx_elem
        x_nodes[k, :] = xL + 0.5 * (xi + 1) * dx_elem

    return {
        'K': K, 'N': N, 'dx_elem': dx_elem,
        'D_phys': D_phys, 'M_local': M_local,
        'x_nodes': x_nodes, 'a': a, 'flux_type': flux_type
    }

def rhs(u_flat, op):
    # puxa os dados do operador DG
    K, N = op['K'], op['N']
    D = op['D_phys']
    M = op['M_local']
    a = op['a']
    flux_type = op['flux_type']

    u = u_flat.reshape((K, N + 1, 2))
    du = np.zeros_like(u)

    # itera sobre cada elemento e vai resolvendo em cada um
    for k in range(K):
        current_x = (k * op['dx_elem']) + xl
        current_eps = 1

        if 0 <= current_x <= 0.5:
            current_eps = 3

```

```

du[k,:,0] = - (D @ u[k,:,1]) / current_eps
du[k,:,1] = - (D @ u[k,:,0]) / mu

# --- Face direita (x_R) ---
if k == K-1:
    Ey_int = u[k, N, 0]
    Hz_int = u[k, N, 1]
    Ey_ext = 0.5 * (Ey_int + Z*Hz_int)
    Hz_ext = 0.5 * (Ey_int + Z*Hz_int) / Z
    EY_L = Ey_int
    HZ_L = Hz_int
    EY_R = Ey_ext
    HZ_R = Hz_ext
else:
    EY_L = u[k, N, 0]
    HZ_L = u[k, N, 1]
    EY_R = u[k+1, 0, 0]
    HZ_R = u[k+1, 0, 1]
if flux_type == 'central':
    EY_star = 0.5*(EY_L + EY_R)
    HZ_star = 0.5*(HZ_L + HZ_R)
else:
    EY_star = 0.5*(EY_L + EY_R) + 0.5*Z*(HZ_L - HZ_R)
    HZ_star = 0.5*(HZ_L + HZ_R) + 0.5/Z*(EY_L - EY_R)

du[k, N, 0] += - (HZ_star - HZ_L) / M[N]
du[k, N, 1] += - (EY_star - EY_L) / M[N]

if k == 0:
    Ey_int = u[k, 0, 0]
    Hz_int = u[k, 0, 1]
    Ey_ext = 0.5 * (Ey_int - Z*Hz_int)
    Hz_ext = -0.5 * (Ey_int - Z*Hz_int) / Z
    EY_L = Ey_ext
    HZ_L = Hz_ext
    EY_R = Ey_int
    HZ_R = Hz_int
else:
    EY_L = u[k-1, N, 0]
    HZ_L = u[k-1, N, 1]
    EY_R = u[k, 0, 0]
    HZ_R = u[k, 0, 1]
if flux_type == 'central':
    EY_star = 0.5*(EY_L + EY_R)
    HZ_star = 0.5*(HZ_L + HZ_R)
else:
    EY_star = 0.5*(EY_L + EY_R) + 0.5*Z*(HZ_L - HZ_R)
    HZ_star = 0.5*(HZ_L + HZ_R) + 0.5/Z*(EY_L - EY_R)

# Correção no primeiro nó (sinal positivo)
du[k, 0, 0] += (HZ_star - HZ_R) / M[0]
du[k, 0, 1] += (EY_star - EY_R) / M[0]

return du.flatten()

# low storage Range Kutta quarta ordem
def lsrk4_step(u, rhs_func, dt, op):
    q = np.zeros_like(u)

```

```

    for i in range(5):
        q = A[i] * q + dt * rhs_func(u, op)
        u = u + B[i] * q

    return u

def DGTD(K, N, CFL, T_final, a, flux_type):
    # gera o operador
    dg_operator = build_dg_operator(K, N, a, flux_type)
    x_nodes = dg_operator['x_nodes']

    # discretização
    dx = dg_operator['dx_elem']
    dt = CFL * dx / (np.abs(a) * (N + 1) ** 2)
    Nt = max(1, int(T_final / dt))
    dt = T_final / Nt # evita erro de arredondamento

    # Condição inicial
    Ey_0 = np.exp(-0.5 * ((x_nodes - ((dx * 3) + x_l)) / 0.08)**2)
    Hz_0 = Ey_0 / Z
    u = np.stack([Ey_0, Hz_0], axis=-1).flatten().copy()

    # snapshots
    u_history = []
    snapshots_list = np.linspace(0, Nt - 10, 6)

    print("INICIANDO MARCHA NO TEMPO")
    print(f"dx={dx:.3e}, dt={dt:.3e}, Nt={Nt}")

    # marcha no tempo
    for step in range(Nt):
        u = lsrk4_step(u, rhs, dt, dg_operator)

        if step in snapshots_list:
            u_history.append(u.reshape((K, N+1, 2)))

    return u_history, dx, dt, snapshots_list

# --- Parâmetros de simulação ---
K = 64 # numero de elementos
N = 4 # ordem polinomial dentro dos elementos
CFL = 0.4
T_final = 3

u_history, dx, dt, snapshots_list = DGTD(K, N, CFL, T_final, vp, 'central')

```